

MSc 2025-2026 · Data Architecture for AI

Labo Santé

Architecture de données containerisée pour un projet IA

✓ **LIVRABLE FINAL** — Sessions 1 + 2 + 3

Aline Maire · 16 juin 2026

Repo : github.com/Almaire-Lab/lab0-sante

Sommaire

1. **Cahier des charges** — contexte, besoins fonctionnels & techniques, périmètre
2. **Schéma SQL** (PostgreSQL) — 5 tables, contraintes, index, export dbdiagram.io
3. **Schéma NoSQL** (MongoDB) — document `consultations` + justification
4. **Diagramme d'architecture** — 6 services, réseaux, volumes
5. **Extraits commentés du `docker-compose.yml`**
6. **Pipeline d'ingestion Python** (Session 2)
7. **Benchmarks de volumétrie** — 60k / 600k / 6M lignes
8. **Monitoring** (Session 3) — Portainer + `docker stats`
9. **Documentation d'utilisation**

1. Cahier des charges

1.1 Contexte

Un **laboratoire de santé fictif** souhaite centraliser ses données médicales aujourd'hui éclatées entre plusieurs fichiers et logiciels métier. L'objectif : bâtir une plateforme **fiable, scalable, administrable, observable** et **portable**. À terme, ces données alimenteront des modèles IA (prédiction de prescription, détection d'interactions médicamenteuses, NLP sur consultations).

1.2 Données à gérer

Domaine	Type	Stockage	Justification
Médecins / Patients / Médicaments	structuré	PostgreSQL	schéma stable, intégrité forte
Prescriptions (N-N)	structuré	PostgreSQL	relations patient ↔ médecin ↔ médicament
Consultations	semi-structuré	MongoDB	symptômes/diagnostics à longueur et structure variables

1.3 Besoins fonctionnels (F1–F6)

- **F1** — Enregistrer/consulter référentiels via UI (pgAdmin) ✓
- **F2** — Créer des prescriptions liant patient + médecin + 1..N médicaments ✓
- **F3** — Stocker consultations (symptômes libres, diagnostic CIM-10) avec `patient_id` ✓
- **F4** — Ingérer en masse jusqu'à 6 M de lignes pour tester la volumétrie ✓ **(validé)**
- **F5** — Re-exécuter le pipeline sans casser les données (idempotence via ON CONFLICT) ✓
- **F6** — Consulter en temps réel l'état des services (Portainer + docker stats) ✓

1.4 Besoins techniques

Catégorie	Exigence
Conteneurisation	<code>docker compose up -d</code> démarre tous les services
Persistance	volumes Docker pour Postgres et Mongo
Réseau	réseau isolé, communication par nom de service
Secrets	variables via <code>.env</code> (jamais commité)
Ordonnancement	<code>depends_on: service_healthy</code> + healthchecks
Observabilité	pgAdmin, Mongo Express, Portainer, <code>docker stats</code>

2. Schéma SQL (PostgreSQL)

Source dbdiagram.io : <https://dbdiagram.io/d/Labo-Sante-Schema-SQL-6a27e6f88eb8ca4bfe863a2b>

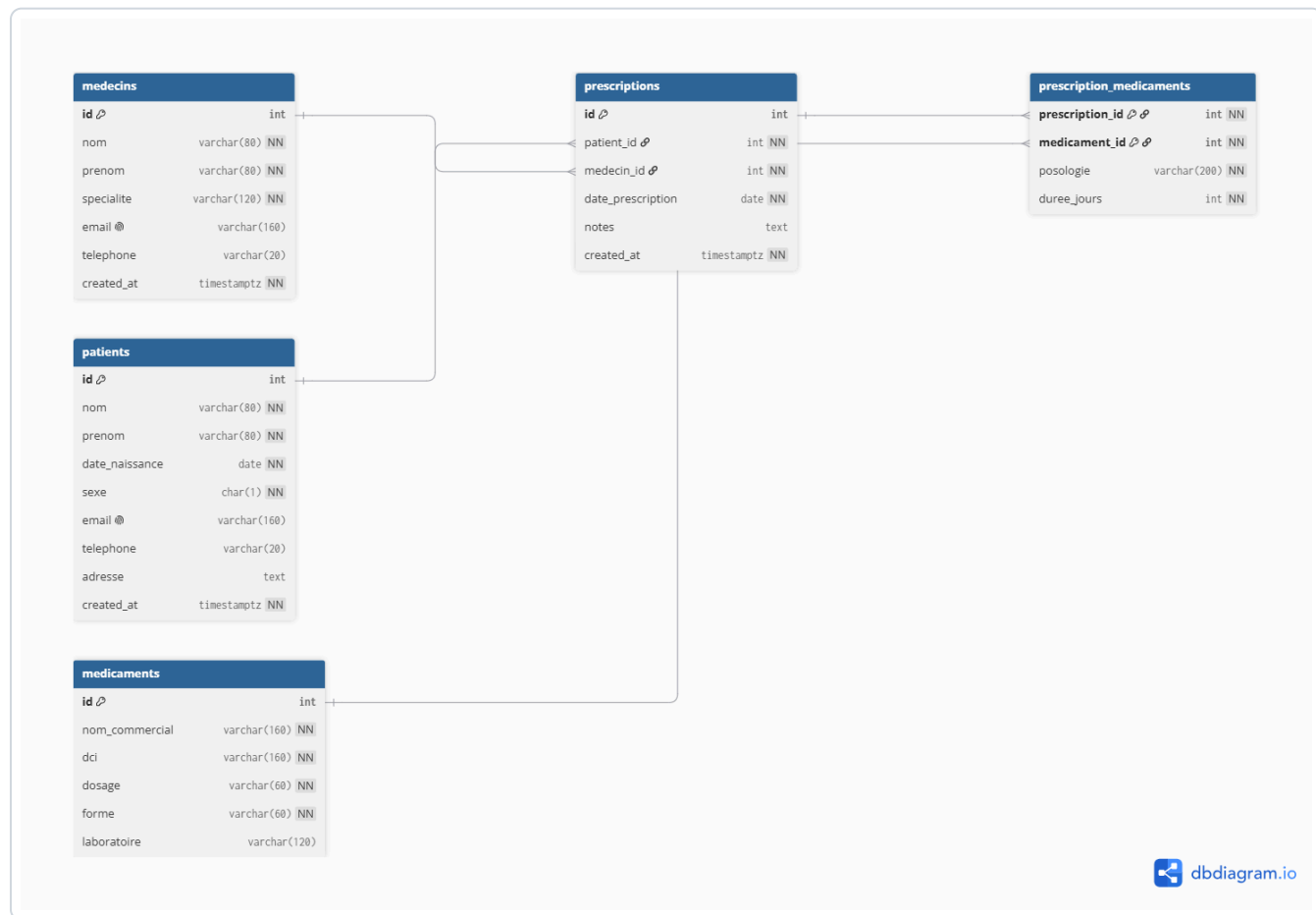


Table	Rôle	Clés
medecins	référentiel praticiens	PK id , UNIQUE email
patients	référentiel patients	PK id , CHECK sexe
medicaments	référentiel	PK + UNIQUE (nom, dosage, forme)
prescriptions	actes de prescription	PK + 2 FK
prescription_medicaments	liaison N-N	PK composite

Index : **patient_id** , **medecin_id** , **date_prescription** , **dci** . Auto-exécution via `/docker-entrypoint-initdb.d/01-schema.sql` .

3. Schéma NoSQL (MongoDB)

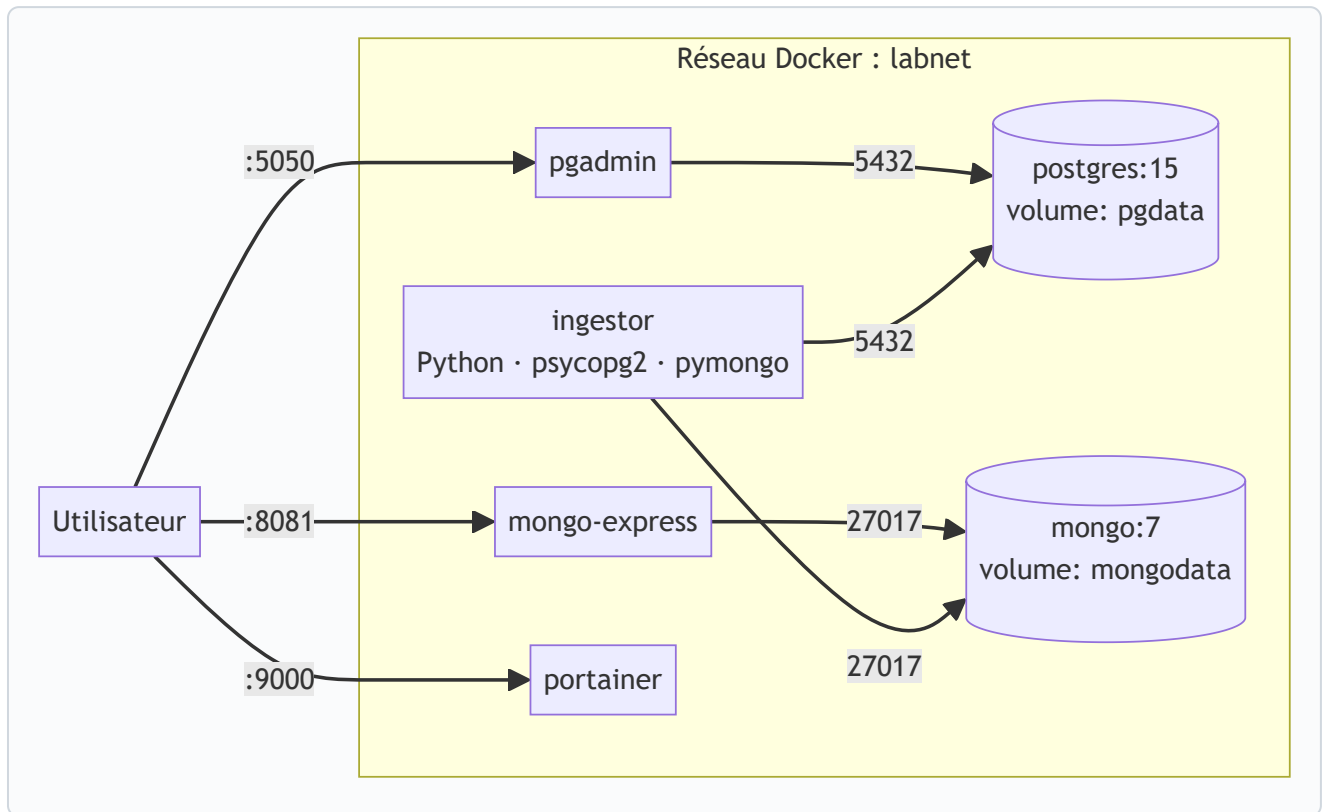
Collection `consultations` . Exemple :

```
{
  "patient_id": 1428, "medecin_id": 12,
  "date": "2026-06-09T11:19:44Z",
  "symptoms": [
    { "label": "toux sèche", "duree_jours": 10 },
    { "label": "fièvre", "temperature_c": 38.4 }
  ],
  "diagnosis": {
    "primary": { "code_cim10": "J20.9", "label": "Bronchite aiguë" },
    "secondary": [{ "code_cim10": "R05", "label": "Toux" }],
    "severity": "moderee"
  }
}
```

Pourquoi NoSQL ici ? Structure variable des symptômes/diagnostics, pas de jointures nécessaires, `patient_id` assure le lien logique avec PostgreSQL, évolution facile sans migration.

Index créés : `patient_id ASC` , `date DESC` , `diagnosis.primary.code_cim10 ASC` .

4. Diagramme d'architecture (6 services)



Service	Image	Port hôte	Volume
postgres	postgres:15-alpine	5432	pgdata
mongo	mongo:7	27017	mongodata
pgadmin	dpage/pgadmin4	5050	pgadmin
mongo-express	mongo-express:1	8081	—
portainer	portainer/portainer-ce	9000	portainer
ingestor (S2)	build ./app (Python 3.12)	—	—

5. Extraits commentés du `docker-compose.yml`

5.1 Service `postgres`

```
postgres:
  image: postgres:15-alpine          # version 15, base alpine légère
  environment:
    POSTGRES_USER:   ${POSTGRES_USER}      # lu depuis .env
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    POSTGRES_DB:     ${POSTGRES_DB}
  ports: [ "127.0.0.1:5432:5432" ]      # local-only
  volumes:
    - pgdata:/var/lib/postgresql/data      # persistance
    - ./sql/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U $$POSTGRES_USER"]
    interval: 5s
    retries: 10
```

5.2 Service `ingestor` (Session 2)

```
ingestor:
  build: ./app                      # image construite localement
  environment:
    POSTGRES_HOST: postgres          # nom du service, pas localhost
    MONGO_HOST: mongo
    NB_PRESCRIPTIONS: ${NB_PRESCRIPTIONS:-60000}
    NB_CONSULTATIONS: ${NB_CONSULTATIONS:-60000}
  depends_on:
    postgres: { condition: service_healthy }
    mongo:    { condition: service_healthy }
  profiles: ["ingest"]              # ne démarre pas par défaut
```

Pourquoi `profiles` ? L'ingestor est un script **one-shot** qui doit s'exécuter à la demande, pas en continu. `profiles` permet de l'exclure du démarrage par défaut et de l'invoquer via `--profile ingest run --rm`.

6. Pipeline d'ingestion Python (Session 2)

6.1 Structure du dossier `app/`

```
app/
├── Dockerfile           # image Python 3.12-slim + dépendances
├── requirements.txt      # psycopg2-binary, pymongo, faker
├── generate_data.py      # générateurs Faker (FR)
├── ingest_postgres.py    # bulk insert via execute_values
├── ingest_mongo.py       # bulk insert via insert_many
└── run_all.py           # pipeline complet
```

6.2 Optimisations utilisées

Technique	Gain
<code>execute_values()</code> (psycopg2)	10-50× plus rapide que executemany
<code>insert_many(ordered=False)</code>	poursuit en cas de doublon
Batches de 5000 lignes	équilibre RAM/commits
<code>ON CONFLICT DO NOTHING</code>	idempotence sur emails uniques
Index créés AVANT ingestion Mongo	évite ré-indexation post-insert
Image <code>python:3.12-slim</code>	~150 MB, démarrage rapide

6.3 Lancement

```
# Test 60k (défaut)
docker compose --profile ingest run --rm ingestor

# Surcharger via variables d'env (PowerShell)
$env:NB_PRESCRIPTIONS = "600000"
$env:NB_CONSULTATIONS = "600000"
docker compose --profile ingest run --rm ingestor
```


7. Benchmarks de volumétrie

7.1 Résultats des 3 paliers

Volume	Total	Postgres	Mongo	Débit Mongo	Mémoire pic
60 000	44 s	30 s	14 s	4 400 docs/s	~ 250 MB
600 000	1 min 33	62 s	31 s	19 300 docs/s	~ 500 MB
6 000 000	18 min 00	12 min	5 min 44	17 400 docs/s	~ 3.5 GB

7.2 Observations

- **Mongo scale linéairement** (~17-20k docs/s constant)
- **Postgres ralentit légèrement** à grosse volumétrie (overhead FK + table N-N + checkpoints WAL)
- **Block I/O Postgres** : 9.16 GB cumulés après 6M (cohérent avec ~6.6M prescriptions + 2.7M lignes de liaison)
- **Ingestor stable** à ~56 MB RAM peu importe le volume (lit/écrit par batchs streamés)

7.3 Volumes finaux en base

t	count
prescriptions	6 660 000
prescription_medicaments	2 665 752
patients	124 999
medecins	500
medicaments	20
consultations (Mongo)	6 660 001

8. Monitoring (Session 3)

8.1 Portainer pendant ingestion 6M

Containers

Container list

Containers Search... x Start Stop Kill Restart Pause Resume

Name	State	Filter	Quick Actions	Stack	Image
labo-mongo	healthy			labo-sante	mongo:7
labo-mongo-express	running			labo-sante	mongo-express:1
labo-pgadmin	running			labo-sante	dpage/pgadmin4:1
labo-portainer	running			labo-sante	portainer/portaine
labo-postgres	healthy			labo-sante	postgres:15-alpine
labo-sante-ingestor-run-614a9...	running			labo-sante	labo-sante/ingestor

Items per page 10

Liste des conteneurs — l'ingestor en running aux côtés des 5 services healthy.

Environment summary

Dashboard

Environment info

Environment local - Standalone 29.5.3

URL /var/run/docker.sock

Tags -

1 Stack

6 Containers 6 running 0 stopped 2 healthy 0 unhealthy

6 Images 3.1 GB

5 Volumes

Dashboard local — 6 containers, 5 volumes, stack **labo-sante**.

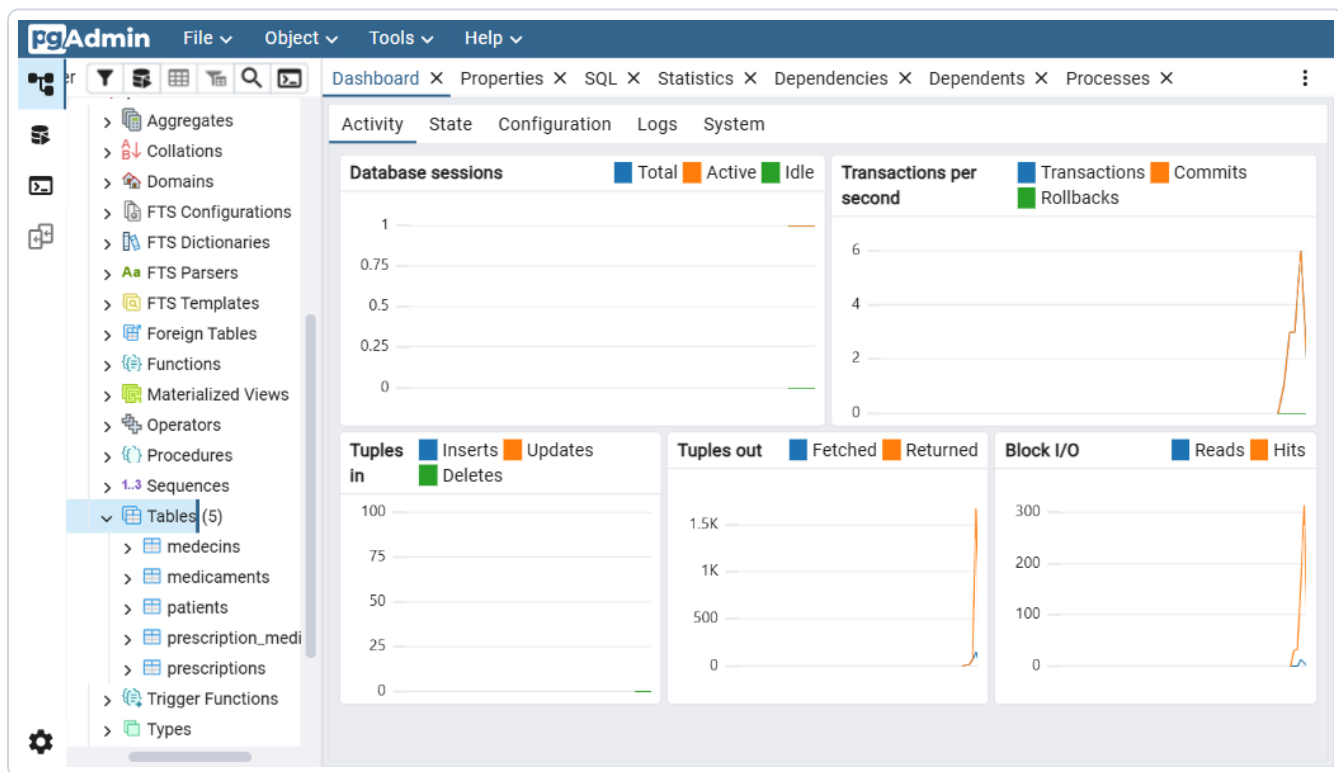
8.2 docker stats pendant phase Postgres du 6M

NAME	CPU %	MEM USAGE	BLOCK I/O
labo-sante-ingestor	25 %	56 MiB	0 B
labo-postgres	75 %	245 MiB	25 MB / 7.33 GB
labo-mongo	0.5 %	498 MiB	287 KB / 286 MB
labo-pgadmin	0.03 %	300 MiB	idle
labo-mongo-express	0 %	47 MiB	idle
labo-portainer	0 %	19 MiB	monitoring

8.3 docker stats après ingestion 6M (au repos)

NAME	CPU %	MEM USAGE	BLOCK I/O
labo-mongo	0.93 %	3.46 GiB	439 MB / 3.47 GB
labo-postgres	0 %	704 MiB	98 MB / 9.16 GB
labo-pgadmin	0.21 %	302 MiB	114 MB / 176 KB
labo-mongo-express	0.10 %	71 MiB	61 MB / 0
labo-portainer	0 %	56 MiB	74 MB / 1 MB

8.4 pgAdmin (UI Postgres)



8.5 Mongo Express (UI Mongo)

PORTAINER.IO

New Portainer installation

Your Portainer instance timed out for security purposes. To re-enable your Portainer instance, you will need to restart Portainer.

For further information, view our [documentation](#).

9. Documentation d'utilisation

9.1 Démarrage rapide

```
# 1. Cloner
git clone https://github.com/Almaire-Lab/labo-sante.git
cd labo-sante

# 2. Configurer les secrets
cp .env.example .env
# (éditer les mots de passe)

# 3. Démarrer la stack (5 services)
docker compose up -d
docker compose ps          # tous doivent être Up (postgres + mongo healthy)

# 4. Vérifier le schéma SQL
docker exec labo-postgres psql -U lab_admin -d labo -c "\dt"

# 5. Lancer une ingestion (60k par défaut)
docker compose --profile ingest run --rm ingestor

# 6. Pour 600k ou 6M
$env:NBPREScriptions = "6000000"
$env:NBConsultations = "6000000"
$env:NBPatients = "100000"
$env:NBMédecins = "500"
docker compose --profile ingest run --rm ingestor
```

9.2 Accès aux UIs

Service	URL	Login
pgAdmin	http://localhost:5050	PGADMIN_EMAIL / PGADMIN_PASSWORD
Mongo Express	http://localhost:8081	MEX_USER / MEX_PASSWORD
Portainer	http://localhost:9000	à créer au 1er accès

9.3 Arrêt / nettoyage

```
docker compose down          # garde les volumes
docker compose down -v       # ⚠ supprime les volumes (perte de données)
```

9.4 Connexion programmatique

```
# Depuis l'extérieur (PowerShell, code Python local...)
postgresql://lab_admin:****@localhost:5432/labo
mongodb://lab_admin:****@localhost:27017/?authSource=admin

# Entre conteneurs (réseau labnet)
postgresql://lab_admin:****@postgres:5432/labo
mongodb://lab_admin:****@mongo:27017/?authSource=admin
```

 **Conclusion** Les 3 sessions du TP sont complétées et validées :

- Session 1 : modélisation + 5 conteneurs UP
- Session 2 : pipeline Python opérationnel, testé à 60k / 600k / **6M lignes**
- Session 3 : monitoring complet via Portainer + `docker stats`

L'infrastructure est **prête pour la suite** : connexion de modèles IA, mise en prod sur Kubernetes, ajout d'une API REST, etc.